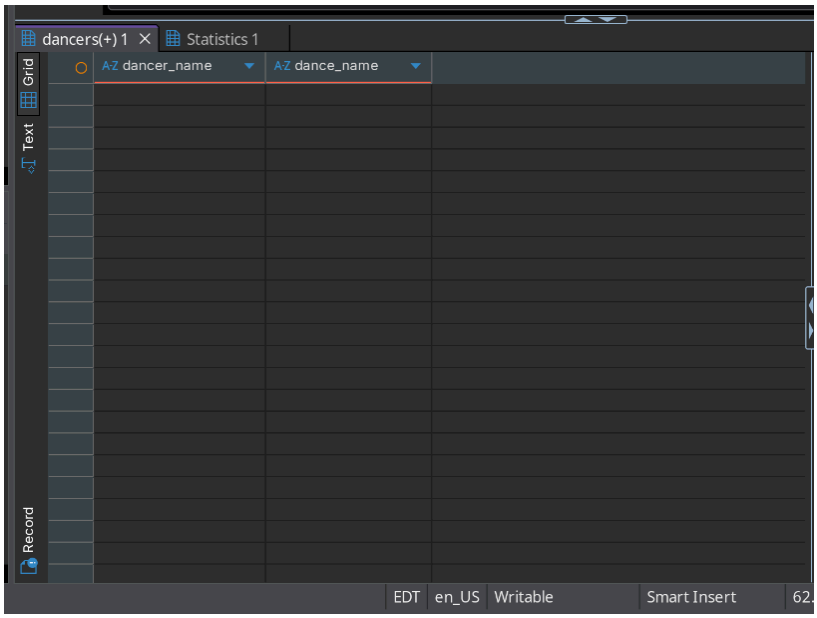
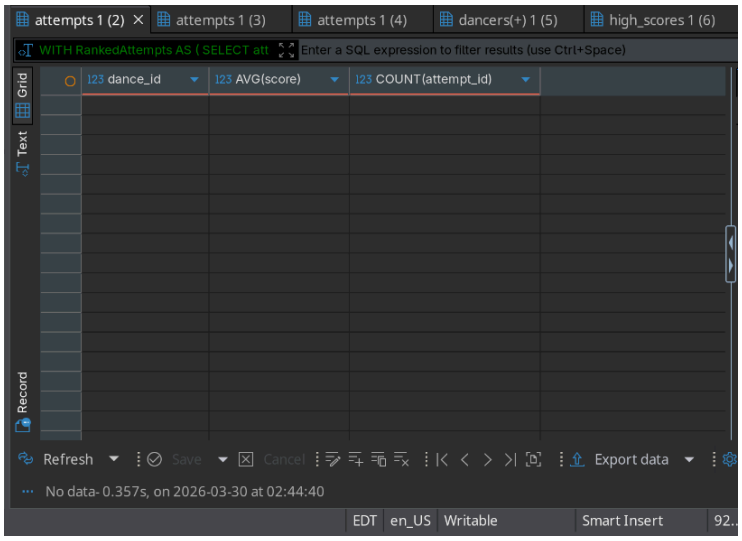
1. Multi join: This query joins the dancer_name and dance_name columns from the dancers and dancer_dances tables. Since there is a junction table (many to many), I first matched dancers.dancer_id to dancer_dances.dancer_id. Then I matched dancer_dances.dance_id to dances.dance_id.
   Tables: dancers, dancer_dances, dances
   Attributes: dancers.dancer_name, dancers.dancer_id, dancer_dances.dancer_id, dancer_dances.dance_id, dances.dance_id, dances.dance_name



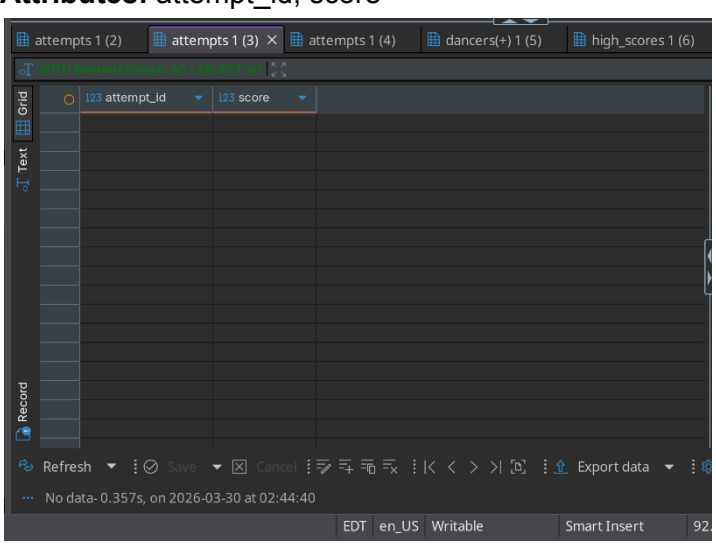

```
SELECT
    dance_id,
    AVG(score),
    COUNT(attempt_id)
FROM attempts
GROUP BY dance_id;
```

2. Group by: The query finds the average score and attempts for each dance. To do this, first select dance_id, average score, and attempt count. Next, I group them by dance_id.
   Tables: attempts
   Attributes: dance_id, score, attempt_id




```
SELECT
   attempt_id,
   score
FROM attempts
WHERE score > (
   SELECT AVG(score)
   FROM attempts
);
```

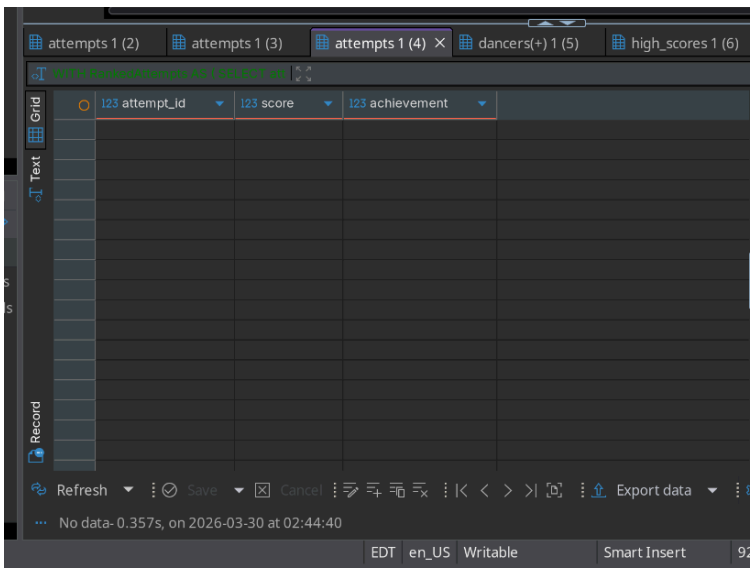
3. Subquery: This query retrieves all attempts with an above average score. I selected attempt_id and score. I then added a where clause to ensure the score is above average. Since you can't do WHERE score > AVG(score), I had to perform a subquery to avoid hardcoding a number.
   Tables: attempts
   Attributes: attempt_id, score




```
SELECT
```

```
SELECT
   attempt_id,
   score
FROM attempts
WHERE score > (
   SELECT AVG(score)
   FROM attempts
);
```

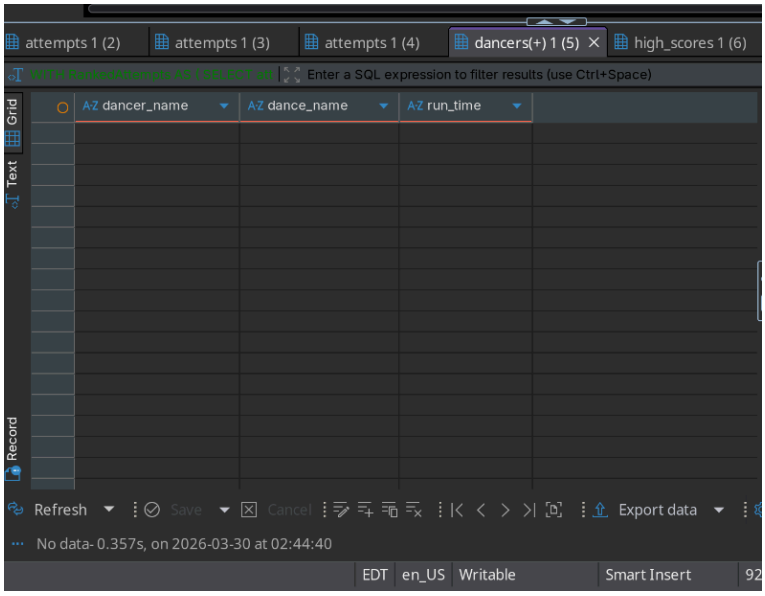
4. Case: This query categorizes each attempt into a poor, average, or great score. I used the case statement to perform the equivalent of an if else on the scores. < 50 = poor score, >=50 <=75 = average score, >75 = great score.
   Tables: attempts
   Attributes: attempt_id, score, achievement




```
SELECT
   attempt_id,
   score,
   CASE
      WHEN score < 50 THEN 'poor score'
      WHEN score BETWEEN 50 AND 75 THEN 'average score'
      ELSE 'great score'
   END AS achievement
FROM attempts;
```

5. Query with view: This query creates a virtual table/view that combines dancer names and dance names. It then queries that view for any dance under 3 minutes. I created a view from a multi-join just like in part 1 (match to the junction then to dances). Then I select any row where run_time is less than 3:00.
   Tables: dancers, dances, dancer_dances

Attributes: dancer_name, dance_name, run_time

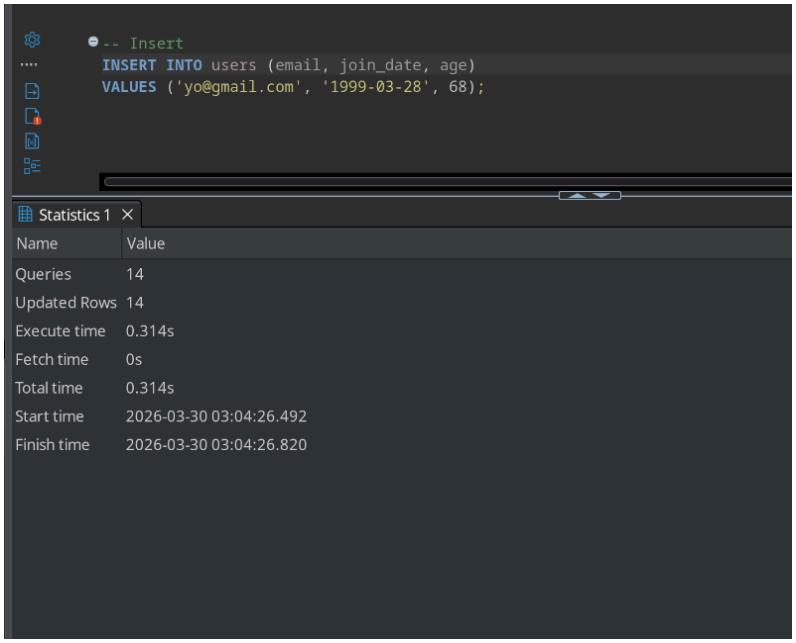



```
CREATE VIEW DancerDances AS
SELECT
   dancers.dancer_name,
   dances.dance_name,
   dancer_dances.run_time
FROM dancers
JOIN dancer_dances
   ON dancers.dancer_id = dancer_dances.dancer_id
JOIN dances
   ON dancer_dances.dance_id = dances.dance_id;

SELECT * FROM DancerDances
WHERE run_time < '00:03:00';
```

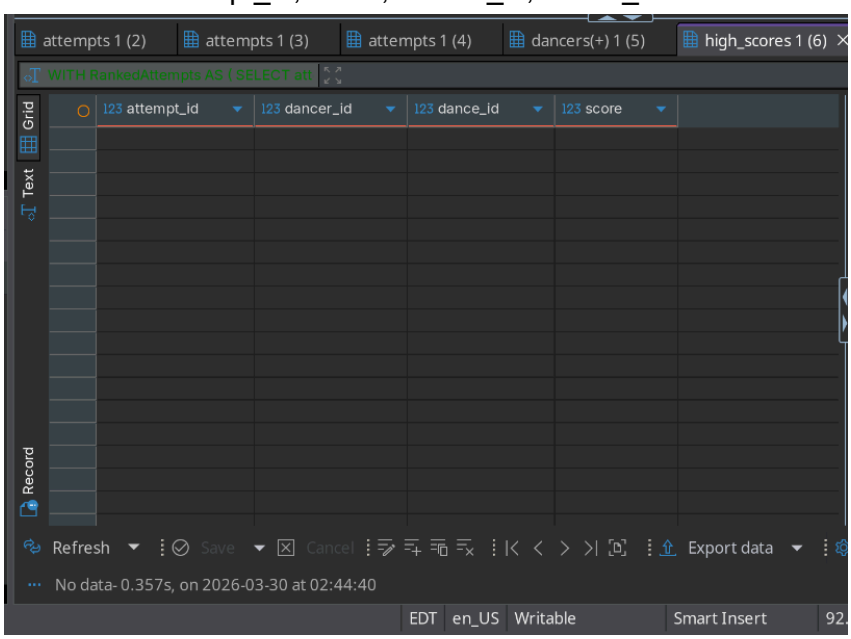
6. Insert: This statement adds a new user to the database. I made sure to add a valid gmail, join date, and age
   Tables: users

Attributes: email, join_date, age




```
INSERT INTO users (email, join_date, age)
VALUES ('yo@gmail.com', '1999-03-28', 68);
```

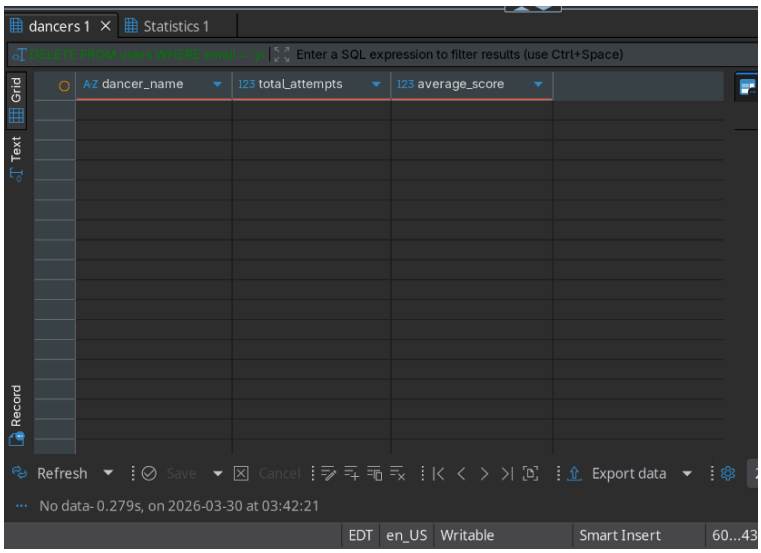
7. Query with filter: This query filters high_scores for entries where dancer_id is 5 and dance_id is 2. I used two conditions in the WHERE clause linked by an AND to ensure both conditions are met.
   Tables: high_scores
   Attributes: attempt_id, score, dancer_id, dance_id




```
SELECT
    attempt_id,
```

```
    dancer_id,
    dance_id,
    score
FROM high_scores
WHERE dancer_id = 5
    AND dance_id = 2;
```

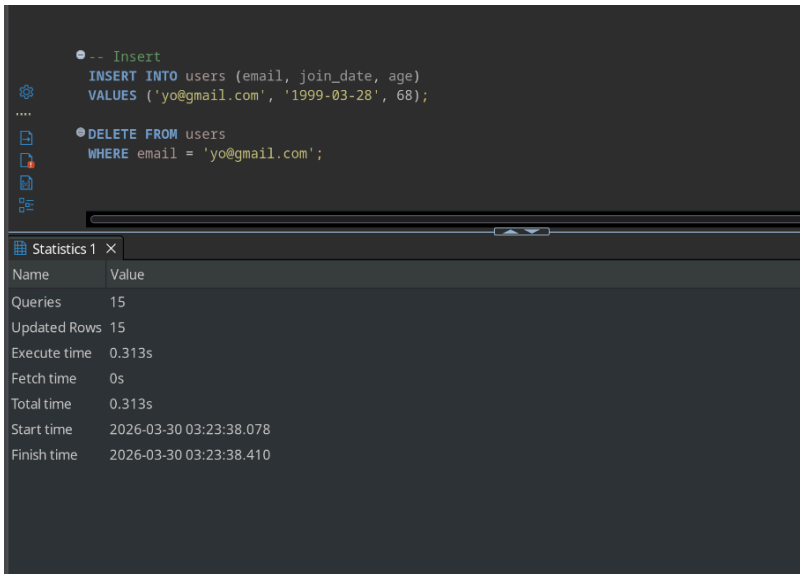
8. Query of choice: This query links all dancers to every attempt made. It then groups the data into one row per dance, containing the average of their scores. Next, it removes any dancer below the score 85. Finally, it places the remaining dancers in descending order by score. For this statement, I used aliases to make it more readable. I
   Tables: dancers, attempts
   Attributes: dancer_id, dancer_name, attempt_id, score




```
SELECT
    d.dancer_name,
    COUNT(a.attempt_id) AS total_attempts,
    ROUND(AVG(a.score), 2) AS average_score
FROM dancers d
JOIN attempts a
    ON d.dancer_id = a.dancer_id
GROUP BY
    d.dancer_id,
    d.dancer_name
HAVING
    AVG(a.score) > 85
ORDER BY
    average_score DESC;
```

9. Delete: deletes the user with email "yo@gmail.com" which we added earlier.

Tables: users
Attributes: email



DELETE FROM users
WHERE email = 'yo@gmail.com';